

What to observe:

- Move your hand toward and away from the HC-SR04 and watch the distance update.
- Verify the zone, motor, and alarm fields change at the correct thresholds.
- A reading of 0.0 cm means no echo was received (object too far or not connected).

Questions for report (Step 1):

1. What is the maximum range of the HC-SR04, and why does `pulseIn()` have a 30 000 μ s timeout? Calculate the corresponding maximum distance.

$$D = (t \cdot 0.034) / 2$$

$$\text{Max } D = (30000 \cdot 0.034) / 2 = 510 \text{ cm}$$

The calculations resulted in a max of 5.1 meters. `pulseIn()` has a 30k microsecond timeout to prevent blocking forever and make use of the full usable sensor range.

2. Why must the TRIG pulse be at least 10 μ s wide? What happens if it is shorter?

The ultra sonic sensor requires a minimum pulse width to trigger the ultrasonic burst. If its shorter the sensor might not fire off, or the sensor will give us 0 cm readings. The pulse ensures the sensor is reliably sending pulses. The shorter the sensor the more unreliable it is as a measurement device.

3. Why does the distance formula divide by 2? What does this represent physically?

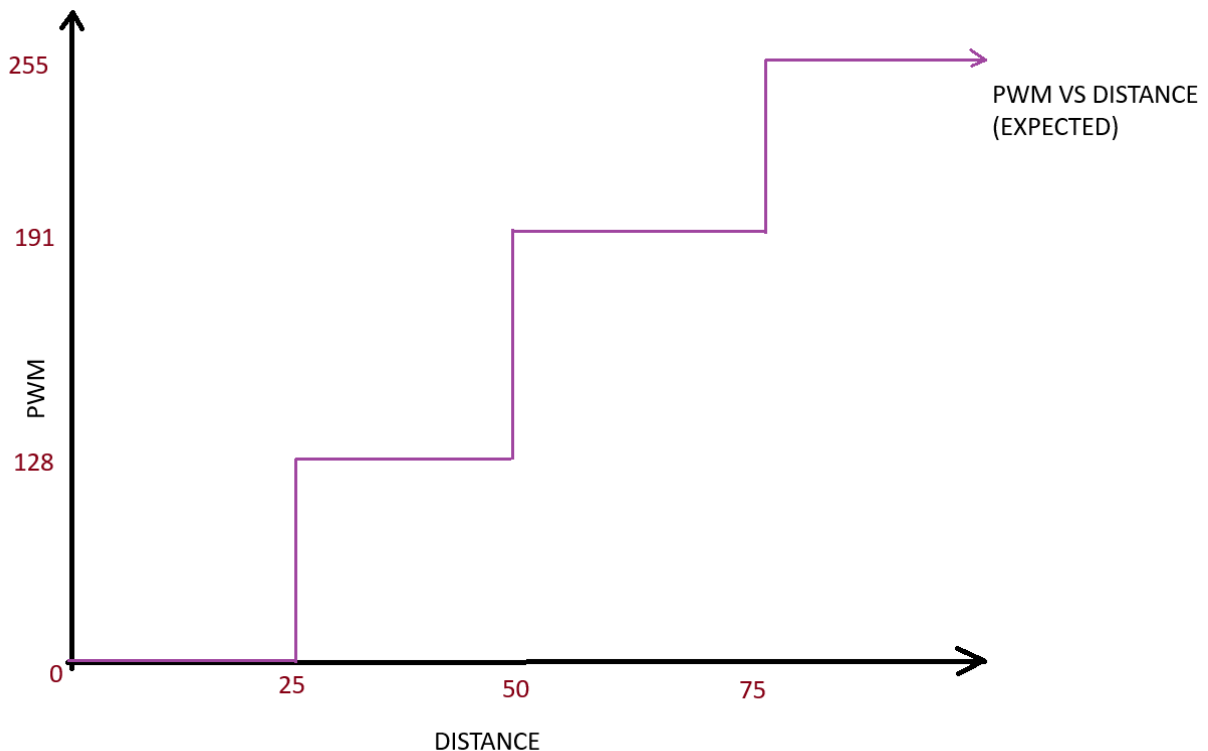
The sound travels from the sensor, to the obstacle, and back to the sensor in a round trip, so it needs to be divided by 2 to convert the round trip time to the one way distance.

What to observe:

- The top panel shows distance with zone color bands (red/orange/yellow/green).
- The dotted red line shows motor PWM dropping as distance decreases.
- The step function is characteristic of zone-based (not continuous) proportional control — this is intentional for this lab.

Questions for report (Step 2):

1. Sketch the expected PWM vs. distance curve. Is this linear? Why or why not?



This is not linear, it is a step graph, as the pwm signal changes immediately once it reaches certain distances. A linear graph would be a smooth line where the pwm gradually increases with distance. The graph here shows discrete pwm levels. This is a step graph.

2. If VCC2 is powered from 5V but the motor needs 9V, what would you expect to observe? How would you fix it?

The motor would run, but be very weak, slow, and will most likely stall under a heavy load. This can be fixed by providing an external 9 volt supply to vcc2 with a common ground with the arduino.

3. Why is data acquisition done in a background thread (separated from plotting)?

This is done to prevent the plot from freezing, or being too slow to collect data. The separation allows the data to be collected and have real time plotting simultaneously.

Questions for report (Step 3):

1. At exactly 50 cm, which zone does the Arduino assign? Does your Python implementation agree? Show the relevant code and explain.

At exactly 50 cm, the zone the arduino assigns is the half speed zone (128). The python implementation agrees with this as shown by the following screen shot:

```
def distance_to_pwm(distance_cm: float) -> int:
    """
    Map a distance measurement to a motor PWM value (0-255).

    The motor speed is inversely proportional to distance - when an object is
    far away the motor runs faster; as it approaches the motor slows and stops.

    Zone boundaries (must match the Arduino firmware exactly):
    distance > 75 cm -> PWM = 255 (Full speed)
    50 < distance ≤ 75 -> PWM = 191 (3/4 speed)
    25 < distance ≤ 50 -> PWM = 128 (Half speed)
    distance ≤ 25 cm -> PWM = 0 (Stop)

    Args:
        distance_cm: Distance in centimetres (float, ≥ 0).

    Returns:
        Integer PWM value in range 0-255.
    """
    if distance_cm > 75:
        return 255
    elif distance_cm > 50:
        return 191
    elif distance_cm > 25:
        return 128
    else:
        return 0
```

2. What would change in both the Arduino and Python code to add a fifth zone (e.g., 75–100 cm at 75% speed)? A new condition statement and a new pwm value would need to be added. The logic structure would be the same, just with these additions.
3. Why is zone-based control simpler to implement than continuous proportional control? What are the trade-offs? The pros of zone based control are:

Its easier to control, you dont need to do math, and the behavior of the pwm is stable.

The cons are:

The transitions are not smooth, there will be sudden jumps in speed, and the control will be less precise.

Questions for report (Step 4):

1. At 60 cm exactly, what alarm state does your function return? What about 19.9 cm?

The state is silent at 60 cm. At 19.9 the state returned is the beeping state.

2. Why does the beeping frequency *increase* as the object gets *closer*? How does this affect the user experience?

The beeping frequency increases as an object gets closer because it is meant to warn the user of an incoming impact. The higher the frequency, the more close the obstacle is. This makes it easier for the user to determine distance and how to respond.

3. Describe a scenario where a fixed-frequency beep (no frequency mapping) would be sufficient. When is variable frequency important?

Its good for simple alarms where there are only two states you need to be worried about, like for instance: an unbuckled seatbelt or an open car door. Variable frequency is more useful in situations where you want feedback on how close something is, like say the rearview parking assistance camera, or a sensor to alert the driver when someone is passing them on an adjacent lane.